PES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC YEAR 2025-2026 | FIFTH SEMESTER B.TECH

CODE COVERAGE AND BRANCH COVERAGE REPORT

(Statement, Line, Function and Branch Coverage of the Unit Test Suite)

PROJECT: SPRINTFLOW – PROJECT MANAGEMENT SYSTEM

(Workflow, Visibility, Estimation, and Kanban)

SUBMITTED BY: TEAM 11 (SECTION 5C)

Student Name	USN	Testing Role (STP Section 9)	Report Owner For
Darshan P Pawar	PES2UG24CS143	Test Architect	Branch Coverage Report (#94)
Dhatri Shivaprasad	PES2UG24CS158	Test Engineer (Frontend)	System Validation Report (#92)
Chandan Kumar K	PES2UG24CS128	Test Engineer (Backend)	Code Coverage Report (#93)
Gurubelli Yekambar Eshwar Rao	PES2UG24CS173	QA Lead	Unit Test Report (#91)

Version: 1.0 (Final Submission) | Date: 06-11-2026 | Testing Phase: 02-11-2026 to 06-11-2026

Repository: https://github.com/Darshanpawar7/SE-SRS

Document: Code Coverage Report and Branch Coverage Report, Testing Phase (Part 3, items 2.b.iii and 2.b.iv)

Report owners: Part A, Code Coverage: Chandan Kumar K, Test Engineer (Backend) (GitHub issue #93). Part B, Branch Coverage: Darshan P Pawar, Test Architect (GitHub issue #94)

Prepared by: Team 11 (Section 5C, PES University)

Date: 06-11-2026

Code under test: `main` branch, commit `f9e0727`

Repository location: `docs/test-reports/CODE_COVERAGE_REPORT.md`, `BRANCH_COVERAGE_REPORT.md`, and the HTML report in `docs/test-reports/coverage-html/`

Revision History

Version	Date	Author	Change Summary
1.0	06-11-2026	Team 11	Statement, line and function coverage (Part A) and branch coverage (Part B) of the whole src folder, measured against the STP target, with gaps and recommended tests

1. Introduction

1.1 Purpose

This document reports how much of SprintFlow's source code the automated unit and integration suite executes. Part A gives statement, line and function coverage. Part B gives branch coverage: whether each possible outcome of every decision in the code was taken. Both parts use the same measurement run and are compared with the coverage target in the Software Test Plan (STP Section 14.1).

1.2 Scope

We measured every JavaScript and JSX file in `src/`, 62 files in all, and not only the files a test happens to import. A file that no test touches therefore counts as 0% instead of dropping out of the totals. The test suite is the one described in the Unit Test Report: 35 test files and 234 tests.

1.3 References

- Software Test Plan, STP v1.0 (`docs/STP_Document_Team11.md`), Section 14.1
- Unit Test Report and System Validation Report (`docs/test-reports/`)
- Coverage configuration in `vite.config.js`

2. Method

Item	Value
Code under test	`main` at commit `f9e0727`
Coverage tool	`@vitest/coverage-v8` 4.1.12. It uses the V8 engine's built-in coverage counters and maps them back to the source lines; the HTML report is produced by istanbul.
Included	`src/**/*.{js,jsx}`
Excluded	`src/tests/**` (the tests themselves) and `src/main.jsx` (the 23-line browser entry point that mounts the app)
Command	`npm run test:coverage` (runs `vitest run --coverage`)
Outputs	HTML report (`coverage-html/index.html`) and totals (`evidence/coverage-summary.json`)

Item	Value
Re-check	The measurement was repeated on 06-11-2026 at 21:34 IST before submission and gave identical totals for all four measures.

2.1 What the measures mean

Measure	Meaning
Statement coverage	Share of executable statements that ran at least once.
Line coverage	The same, counted by source line.
Function coverage	Share of functions that were called at least once.
Branch coverage	Share of decision outcomes that were taken at least once (Part B).
Decision point	One place in the code where execution can go more than one way.
Outcome	One way a decision can go. An `if` has two (including an implicit empty `else`); `a ?? b` has two; `a && b && c` has three; a default parameter has one (the default is used).

Several Sprint 2 files put many statements on one long line; `ProjectMap.jsx`, for example, has 60 statements on 24 lines. For files like these, statement coverage is the fairer of the first two measures.

2.2 Note on the run

Coverage instrumentation slows every statement down. Under it, the Kanban timing benchmark (`kanbanPerformance.test.jsx`, 50 ms budget) measured 66 ms and failed. The run is configured to report coverage anyway (`reportOnFailure` in `vite.config.js`). The benchmark's code still ran, so it still counts towards coverage. The pass/fail result of that test comes from the normal run, where it passes at 13.7 ms (Unit Test Report, Section 8).

3. Summary Against the Test Plan Target

The STP sets one coverage target (Section 14.1): "Code Coverage (Unit Tests): percentage of utility module code exercised by Vitest, 80% or higher."

Scope	Files	Statements	Lines	Functions	Branches	Target	Result
Utility modules (`src/utils`)	26	**98.5%**	**99.3%**	**99.1%**	92.9%	80% or higher	**Met**
Logic layer (`src/utils` + `src/services`)	32	93.5%	95.2%	94.9%	87.9%	–	–
Whole application (`src/`)	62	66.3% (1412/2130)	70.5% (1098/1558)	56.6% (421/744)	64.1% (995/1552)	–	–

Verdict: the STP coverage target is met by a wide margin. The business logic is close to fully covered: estimation, workflow, burndown, audit trail, filtering and the sprint rules. The lower whole-application figure comes from React view components. The unit suite renders only some of those components; the rest are checked end to end by the system test cases (Section 6.2).

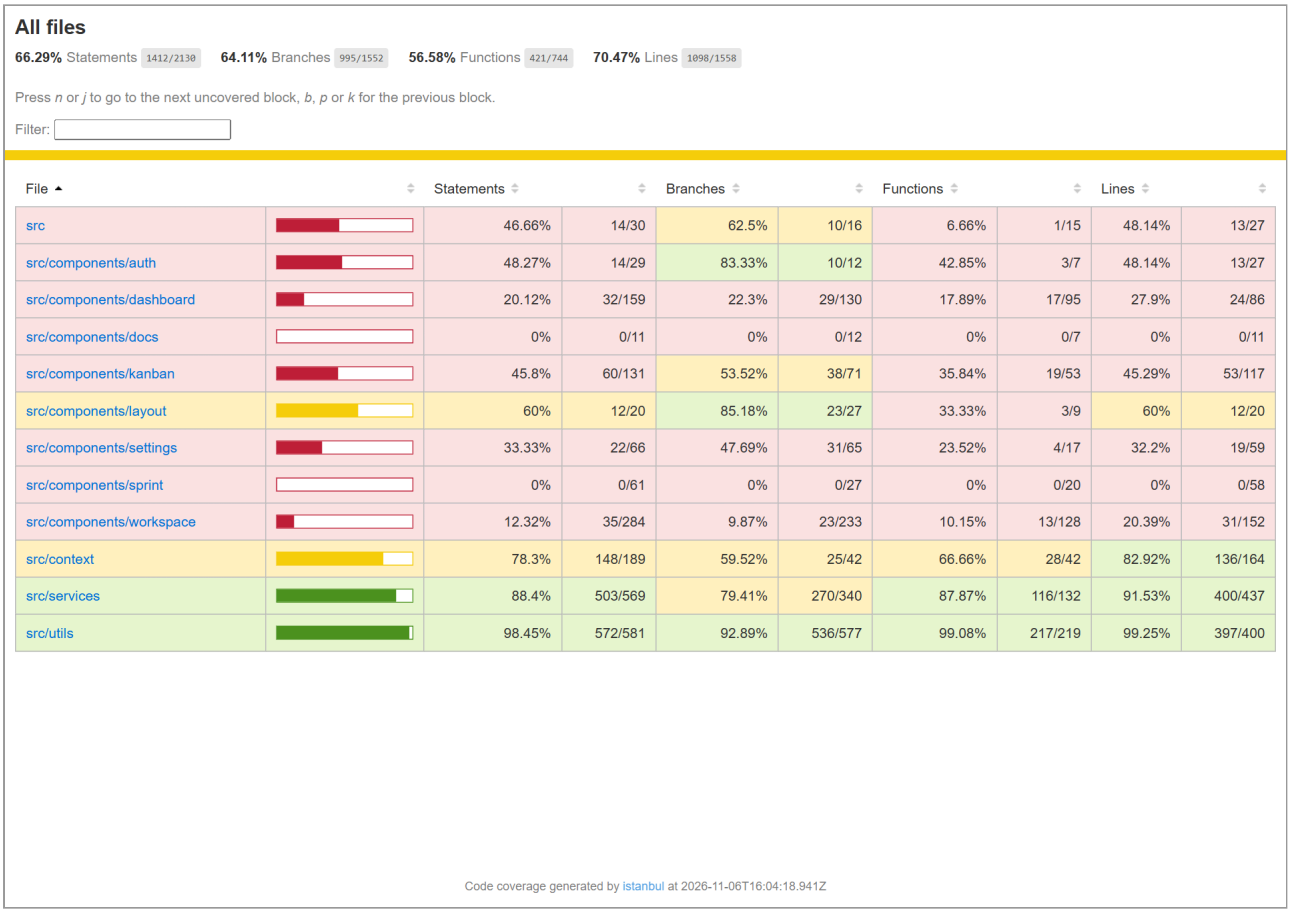


Figure 1: Coverage summary page of the HTML report (istanbul), all folders of src/.

4. Coverage by Folder

Folder	Files	Statements	Branches	Functions	Lines
`src/App.jsx`	1	46.7%	62.5%	6.7%	48.1%

Folder	Files	Statements	Branches	Functions	Lines
src/components/auth	1	48.3%	83.3%	42.9%	48.1%
src/components/dashboard	8	20.1%	22.3%	17.9%	27.9%
src/components/docs	1	0.0%	0.0%	0.0%	0.0%
src/components/kanban	3	45.8%	53.5%	35.8%	45.3%
src/components/layout	2	60.0%	85.2%	33.3%	60.0%
src/components/settings	2	33.3%	47.7%	23.5%	32.2%
src/components/sprint	1	0.0%	0.0%	0.0%	0.0%
src/components/workspace	9	12.3%	9.9%	10.2%	20.4%
src/context	2	78.3%	59.5%	66.7%	82.9%
src/services	6	88.4%	79.4%	87.9%	91.5%
src/utils	26	98.5%	92.9%	99.1%	99.3%
All of src/	**62**	**66.3%**	**64.1%**	**56.6%**	**70.5%**

5. Coverage by File

5.1 Logic layer: utilities, services and contexts

File	Statements	Branches	Functions	Lines
context/AuthContext.jsx	61.5% (8/13)	n/a (0/0)	66.7% (4/6)	58.3% (7/12)
context/ProjectContext.jsx	79.5% (140/176)	59.5% (25/42)	66.7% (24/36)	84.9% (129/152)
services/authService.js	93.0% (40/43)	89.3% (25/28)	100.0% (7/7)	91.7% (33/36)
services/feedbackStore.js	70.6% (24/34)	20.0% (2/10)	55.6% (5/9)	63.6% (7/11)
services/flowHistoryStore.js	17.9% (5/28)	0.0% (0/23)	0.0% (0/7)	30.8% (4/13)
services/localPersistence.js	93.8% (15/16)	91.7% (11/12)	100.0% (4/4)	91.7% (11/12)
services/mockData.js	100.0% (9/9)	n/a (0/0)	100.0% (1/1)	100.0% (8/8)
services/supabaseClient.js	93.4% (410/439)	86.9% (232/267)	95.2% (99/104)	94.4% (337/357)
utils/accessibility.js	100.0% (7/7)	100.0% (6/6)	100.0% (1/1)	100.0% (5/5)
utils/auditTrail.js	98.0% (50/51)	92.1% (58/63)	100.0% (18/18)	100.0% (42/42)
utils/burndown.js	100.0% (55/55)	100.0% (33/33)	100.0% (13/13)	100.0% (41/41)
utils/commands.js	100.0% (8/8)	83.3% (5/6)	100.0% (4/4)	100.0% (6/6)
utils/constants.js	100.0% (4/4)	n/a (0/0)	n/a (0/0)	100.0% (4/4)
utils/deliverySignals.js	100.0% (33/33)	97.0% (32/33)	100.0% (17/17)	100.0% (24/24)
utils/estimation.js	100.0% (30/30)	77.3% (17/22)	100.0% (18/18)	100.0% (23/23)
utils/flowAnalytics.js	98.1% (53/54)	100.0% (35/35)	95.0% (19/20)	100.0% (29/29)
utils/kanban.js	100.0% (4/4)	100.0% (1/1)	100.0% (3/3)	100.0% (3/3)
utils/kpis.js	100.0% (2/2)	n/a (0/0)	100.0% (1/1)	100.0% (2/2)
utils/labels.js	100.0% (13/13)	100.0% (4/4)	100.0% (7/7)	100.0% (10/10)
utils/projectMap.js	100.0% (43/43)	100.0% (24/24)	100.0% (17/17)	100.0% (26/26)
utils/projectMetadata.js	100.0% (27/27)	87.1% (27/31)	100.0% (4/4)	100.0% (24/24)
utils/savedWorkViews.js	100.0% (30/30)	88.5% (23/26)	100.0% (12/12)	100.0% (18/18)
utils/sprintLifecycle.js	100.0% (27/27)	100.0% (28/28)	100.0% (8/8)	100.0% (20/20)

File	Statements	Branches	Functions	Lines
utils/sprintReports.js	95.9% (70/73)	88.0% (44/50)	100.0% (37/37)	100.0% (36/36)
utils/storyPoints.js	100.0% (3/3)	100.0% (4/4)	100.0% (1/1)	100.0% (3/3)
utils/taskExplorer.js	92.9% (13/14)	91.2% (62/68)	87.5% (7/8)	90.9% (10/11)
utils/taskFilters.js	100.0% (13/13)	93.3% (14/15)	100.0% (4/4)	100.0% (10/10)
utils/taskForm.js	94.4% (17/18)	80.0% (24/30)	100.0% (8/8)	94.1% (16/17)
utils/taskUpdates.js	100.0% (13/13)	100.0% (8/8)	100.0% (7/7)	100.0% (10/10)
utils/userText.js	100.0% (8/8)	92.9% (13/14)	100.0% (1/1)	100.0% (6/6)
utils/workflow.js	100.0% (10/10)	100.0% (6/6)	100.0% (3/3)	100.0% (7/7)
utils/workSignals.js	100.0% (9/9)	100.0% (17/17)	100.0% (2/2)	100.0% (5/5)
utils/workspaceFeedback.js	88.9% (16/18)	93.9% (31/33)	100.0% (3/3)	88.9% (8/9)
utils/workViews.js	100.0% (14/14)	100.0% (20/20)	100.0% (2/2)	100.0% (9/9)

5.2 User interface: components and the app shell

File	Statements	Branches	Functions	Lines
App.jsx	46.7% (14/30)	62.5% (10/16)	6.7% (1/15)	48.1% (13/27)
components/auth/SignInView.jsx	48.3% (14/29)	83.3% (10/12)	42.9% (3/7)	48.1% (13/27)
components/dashboard/AuditTrail.jsx	88.9% (8/9)	75.0% (15/20)	80.0% (4/5)	85.7% (6/7)
components/dashboard/BurndownChart.jsx	100.0% (16/16)	100.0% (8/8)	100.0% (9/9)	100.0% (11/11)
components/dashboard/DashboardView.jsx	0.0% (0/13)	0.0% (0/9)	0.0% (0/7)	0.0% (0/11)
components/dashboard/DeliverySignals.jsx	0.0% (0/8)	0.0% (0/15)	0.0% (0/6)	0.0% (0/6)
components/dashboard/FlowAnalytics.jsx	6.3% (3/48)	0.0% (0/32)	0.0% (0/28)	10.5% (2/19)
components/dashboard/KpiCards.jsx	100.0% (2/2)	100.0% (4/4)	100.0% (2/2)	100.0% (2/2)
components/dashboard/ReportsWorkbench.jsx	0.0% (0/60)	0.0% (0/40)	0.0% (0/36)	0.0% (0/27)
components/dashboard/WorkloadList.jsx	100.0% (3/3)	100.0% (2/2)	100.0% (2/2)	100.0% (3/3)
components/docs/DocsView.jsx	0.0% (0/11)	0.0% (0/12)	0.0% (0/7)	0.0% (0/11)
components/kanban/KanbanBoard.jsx	46.9% (23/49)	56.3% (18/32)	32.0% (8/25)	47.6% (20/42)
components/kanban/LabelPills.jsx	100.0% (5/5)	100.0% (2/2)	100.0% (2/2)	100.0% (4/4)
components/kanban/TaskModal.jsx	41.6% (32/77)	48.6% (18/37)	34.6% (9/26)	40.8% (29/71)
components/layout/Navbar.jsx	56.3% (9/16)	82.6% (19/23)	16.7% (1/6)	56.3% (9/16)
components/layout/Sidebar.jsx	75.0% (3/4)	100.0% (4/4)	66.7% (2/3)	75.0% (3/4)
components/settings/ProjectSettingsModal.jsx	47.8% (11/23)	82.6% (19/23)	42.9% (3/7)	45.0% (9/20)
components/settings/SettingsModal.jsx	25.6% (11/43)	28.6% (12/42)	10.0% (1/10)	25.6% (10/39)
components/sprint/SprintManager.jsx	0.0% (0/61)	0.0% (0/27)	0.0% (0/20)	0.0% (0/58)
components/workspace/CommandMenu.jsx	24.5% (12/49)	18.8% (6/32)	20.0% (3/15)	34.3% (12/35)
components/workspace/ErrorBoundary.jsx	14.3% (1/7)	0.0% (0/2)	33.3% (2/6)	16.7% (1/6)
components/workspace/ProjectMap.jsx	0.0% (0/60)	0.0% (0/67)	0.0% (0/29)	0.0% (0/24)
components/workspace/SignalIcon.jsx	100.0% (2/2)	66.7% (2/3)	100.0% (1/1)	100.0% (2/2)
components/workspace/TaskExplorer.jsx	0.0% (0/93)	0.0% (0/35)	0.0% (0/41)	0.0% (0/47)
components/workspace/TaskPreviewDrawer.jsx	7.7% (1/13)	0.0% (0/35)	0.0% (0/9)	20.0% (1/5)
components/workspace/TaskSignals.jsx	100.0% (6/6)	75.0% (9/12)	100.0% (4/4)	100.0% (4/4)
components/workspace/WorkSignals.jsx	100.0% (2/2)	20.0% (2/10)	100.0% (1/1)	100.0% (2/2)
components/workspace/WorkspaceFeedback.jsx	21.2% (11/52)	10.8% (4/37)	9.1% (2/22)	33.3% (9/27)

6. Analysis

6.1 Fully covered files

These 19 files have 100% line coverage and every branch outcome taken:

- `utils`: accessibility, burndown, constants, flowAnalytics, kanban, kpis, labels, projectMap, sprintLifecycle, storyPoints, taskUpdates, workflow, workSignals, workViews
- `services`: mockData
- `components`: BurndownChart, KpiCards, WorkloadList, LabelPills

6.2 Files the unit suite does not execute

Seven view components have 0% unit coverage. All of them render inside the running application, and five are exercised by system test cases.

Component	Lines	How it is verified	Logic it delegates to (unit coverage, lines)
`dashboard/DashboardView.jsx`	11	TC-DASH-01, TC-WORK-01, TC-BURN-01, TC-LOG-01, TC-PROJ-01	`kpis` 100%, `estimation` 100%, `burndown` 100%
`dashboard/DeliverySignals.jsx`	6	Rendered on the Overview in the cases above (no dedicated check)	`deliverySignals` 100%
`docs/DocsView.jsx`	11	TC-DOCS-01 (all three tabs)	Static content
`sprint/SprintManager.jsx`	58	TC-SPR-01, TC-EST-01, TC-EST-02	`sprintLifecycle` 100%, `estimation` 100%
`workspace/TaskExplorer.jsx`	47	Opened in TC-AVAIL-01 (render only)	`taskExplorer` 90.9%, `savedWorkViews` 100%
`dashboard/ReportsWorkbench.jsx`	27	**Not verified at system level** (Sprint 2 enhancement, PR #86)	`sprintReports` 100%
`workspace/ProjectMap.jsx`	24	**Not verified at system level** (Sprint 2 enhancement, PR #89)	`projectMap` 100%

The last two are Sprint 2 enhancements that the Test Case Specification predates. Their calculations are unit-tested through the utilities they call, but the screens themselves have no automated check yet.

6.3 Gaps in the logic layer

File	Statements	Gap
`services/flowHistoryStore.js`	17.9%	Added in PR #86. No test imports it. It keeps the daily flow history in LocalStorage behind an in-memory cache, so reading, recording, corrupt-storage handling and the cache are all untested.
`services/feedbackStore.js`	70.6%	Added in PR #88. Settings storage is tested. `announceSaved`, which builds the "Saved" / "Completed" notice, triggers haptics and sets the auto-dismiss timer, is not, and neither is the subscription.
`context/ProjectContext.jsx`	79.5%	Guard clauses (a stale load, a missing task or sprint) and deleting the active sprint are untested.
`context/AuthContext.jsx`	61.5%	Session restore is tested. The context's `signIn` and `signOut` wrappers are not; the unit tests call `authService` directly. Sign-in is exercised by every system test case.
`services/supabaseClient.js`	93.4%	High overall. The misses are failure exits in the cloud sync queue (see Section 11).

7. Recommendations (Code Coverage)

Priority	Action	Expected effect
1	Add unit tests for `services/flowHistoryStore.js`: empty and corrupt storage, one observation per day, cache invalidation	The only logic-layer file below 50%

Priority	Action	Expected effect
2	Add component tests (React Testing Library in jsdom) for `SprintManager`, `TaskExplorer` and `DashboardView`	Moves the three largest untested views into the unit suite
3	Add system cases for the Reports & analytics tab and the Project map to the Test Case Specification	Closes the two screens with no automated check
4	Run `npm run test:coverage` in CI with per-folder thresholds (for example 90% lines for `src/utils`, 85% for `src/services`), and keep the timing benchmark in a separate step	Stops coverage of the logic layer from silently dropping

PART B: BRANCH COVERAGE REPORT

8. Why Branch Coverage

Statement coverage shows whether a line ran. Branch coverage shows whether each possible outcome of every decision ran: both sides of an `if`, both results of a ternary, each operand of `&&`, `||` and `??`, and every default parameter value. This matters because a decision can execute in every test while one of its outcomes never does. A bug on the untaken side, for example an error path that only runs when the network fails, is then invisible to the suite.

9. Branch Coverage Summary

Scope	Decision points	Outcomes	Outcomes taken	Branch coverage
Utility modules (`src/utils`, 26 files)	294	577	536	**92.9%**
Services (`src/services`, 6 files)	168	340	270	**79.4%**
Logic layer (`src/utils` + `src/services`)	462	917	806	**87.9%**
React contexts (`src/context`, 2 files)	24	42	25	59.5%
Whole application (`src/`, 62 files)	778	1552	995	**64.1%**

Across the application, 438 of 778 decision points (56.3%) have every outcome taken.

Verdict. The STP sets no separate branch target, so we compared branch coverage with its 80% target for utility modules (Section 14.1). The utilities reach 92.9%, so the target is met. The services reach 79.4%, just under it; the missed outcomes there are almost all failure and fallback paths, listed in Section 11. The React components pull the whole-application figure down, for the same reason as in Part A.

10. Branch Coverage by Kind of Decision

Kind of decision	Decision points	Outcomes	Outcomes taken	Branch coverage	Points with every outcome taken
Logical `&&`, `||`, `??`	324	717	452	63.0%	160 (49.4%)
`if` / `else`	206	412	306	74.3%	140 (68.0%)
Ternary `a ? b : c`	175	350	166	47.4%	67 (38.3%)
Default parameter value	73	73	71	97.3%	71 (97.3%)
All	**778**	**1552**	**995**	**64.1%**	**438 (56.3%)**

- **Default parameters** are almost fully covered, since tests rarely leave out an argument by accident.
- **`if` / `else`** is the best-tested real decision, at 74.3%.
- **Ternaries (47.4%)** are mostly conditional rendering in JSX, such as `isOpen ? <Dialog/> : null`. Their coverage follows how much of each component the unit tests render.
- **Logical expressions (63.0%)** are the same, plus `??` and `||` fallbacks for missing fields, which tests seldom reach.

11. Branch Coverage by Folder and Logic-Layer File

Folder	Files	Decision points	Outcomes	Outcomes taken	Branch coverage
src/App.jsx	1	8	16	10	62.5%
src/components/auth	1	6	12	10	83.3%
src/components/dashboard	8	65	130	29	22.3%
src/components/docs	1	6	12	0	0.0%
src/components/kanban	3	34	71	38	53.5%
src/components/layout	2	13	27	23	85.2%
src/components/settings	2	32	65	31	47.7%
src/components/sprint	1	13	27	0	0.0%
src/components/workspace	9	115	233	23	9.9%
src/context	2	24	42	25	59.5%
src/services	6	168	340	270	79.4%
src/utils	26	294	577	536	92.9%

The last column of the next table lists the source lines that have at least one outcome no test takes. Each one is highlighted in the HTML report (Figure 2 shows an example).

File	Decision points	Outcomes taken	Branch coverage	Lines with a missed outcome
context/AuthContext.jsx	0	0/0	n/a	–
context/ProjectContext.jsx	24	25/42	59.5%	79, 81, 83, 93, 120, 146, 165, 178, 188, 193, 196, 206, 223, 235
services/authService.js	14	25/28	89.3%	33, 40, 75
services/feedbackStore.js	5	2/10	20.0%	11, 12
services/flowHistoryStore.js	12	0/23	0.0%	3, 5, 6, 7, 14, 15, 18
services/localPersistence.js	4	11/12	91.7%	9
services/mockData.js	0	0/0	n/a	–
services/supabaseClient.js	133	232/267	86.9%	36, 60, 61, 89, 92, 138, 160, 199, 205, 207, 210, 215, 218, 224, 227, 447, 457, 461, 512, 551, 584, 587, 607, 609, 644
utils/accessibility.js	3	6/6	100.0%	–
utils/auditTrail.js	33	58/63	92.1%	57, 188, 193
utils/burndown.js	18	33/33	100.0%	–
utils/commands.js	3	5/6	83.3%	5
utils/constants.js	0	0/0	n/a	–
utils/deliverySignals.js	17	32/33	97.0%	25
utils/estimation.js	14	17/22	77.3%	11, 14, 34, 37, 60
utils/flowAnalytics.js	18	35/35	100.0%	–
utils/kanban.js	1	1/1	100.0%	–
utils/kpis.js	0	0/0	n/a	–
utils/labels.js	2	4/4	100.0%	–

File	Decision points	Outcomes taken	Branch coverage	Lines with a missed outcome
utils/projectMap.js	11	24/24	100.0%	–
utils/projectMetadata.js	16	27/31	87.1%	7, 15, 23, 43
utils/savedWorkViews.js	11	23/26	88.5%	6, 10, 17
utils/sprintLifecycle.js	12	28/28	100.0%	–
utils/sprintReports.js	27	44/50	88.0%	7, 22, 24, 46, 51
utils/storyPoints.js	2	4/4	100.0%	–
utils/taskExplorer.js	32	62/68	91.2%	9, 12, 13, 14, 15, 26
utils/taskFilters.js	9	14/15	93.3%	10
utils/taskForm.js	15	24/30	80.0%	14, 34, 36, 37, 38, 52
utils/taskUpdates.js	5	8/8	100.0%	–
utils/userText.js	9	13/14	92.9%	13
utils/workflow.js	3	6/6	100.0%	–
utils/workSignals.js	8	17/17	100.0%	–
utils/workspaceFeedback.js	15	31/33	93.9%	7, 13
utils/workViews.js	10	20/20	100.0%	–

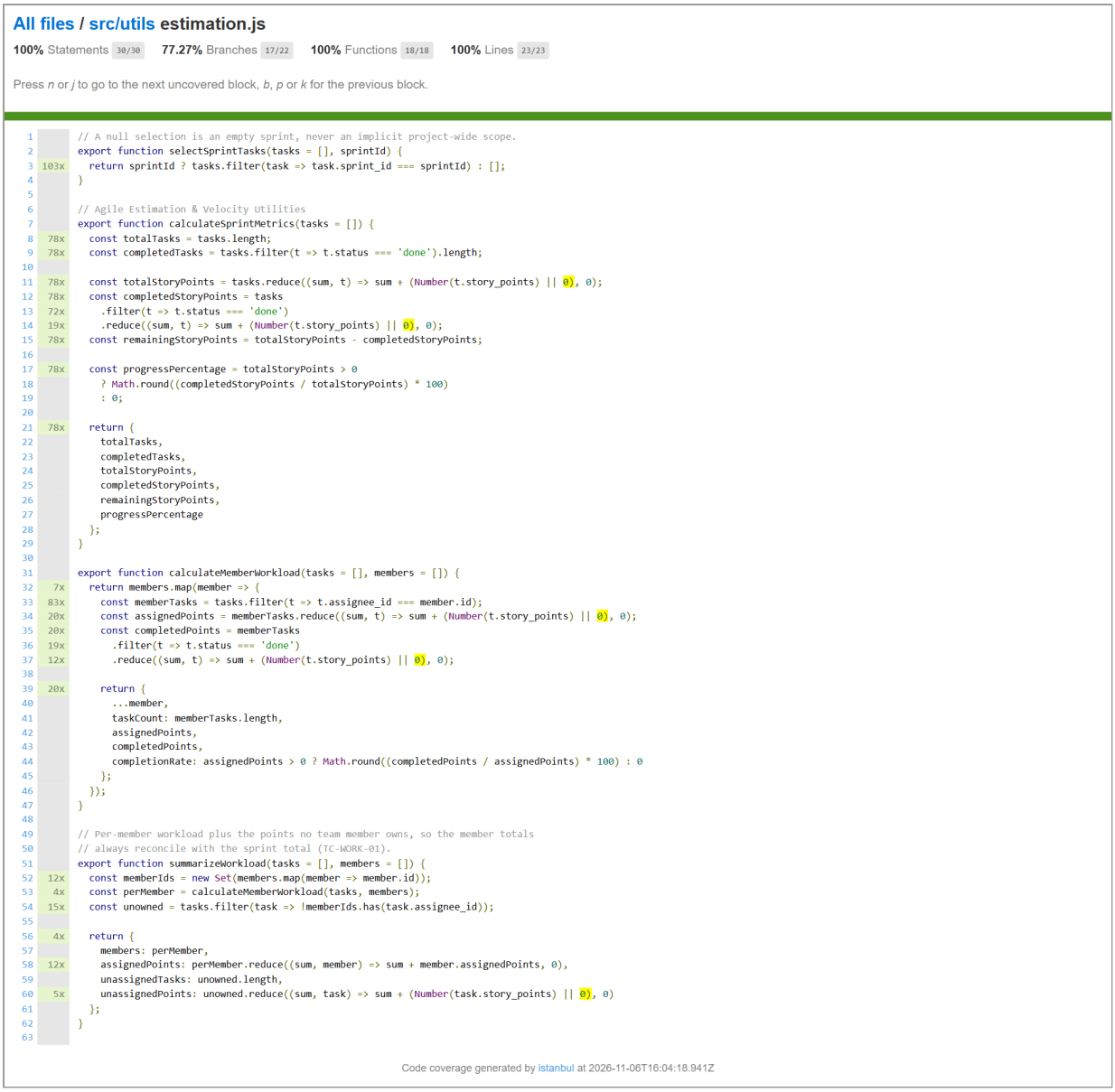


Figure 2: HTML report for src/utils/estimation.js: all statements ran, but the five "|| 0" fallbacks (yellow) were never taken, giving 77.27% branch coverage (17/22).

11.1 What the missed outcomes are

File	Coverage	Missed outcomes, by behaviour
`services/supabaseClient.js`	86.9% (232/267)	**Sync-queue failure exits** (lines 447, 457, 461, 584): when deleting a sprint or inserting an audit row fails in the cloud, the queue stops and keeps the change for retry; untested. **Older audit rows** (lines 199–227): rows saved by earlier builds without metadata, whose type is inferred from the action text. **Read fallbacks** (lines 510–512, 607, 644): a cloud read failing, returning no data, or overtaken by a newer local change. **Defaults for missing values** (lines 36, 60, 138, 160).
`utils/estimation.js`	77.3% (17/22)	The `|| 0` fallback on lines 11, 14, 34, 37 and 60, used when a task's story points are missing or not a number. Every test task has a valid estimate, so the fallback never runs.
`context/ProjectContext.jsx`	59.5% (25/42)	**Guards:** a late load result ignored after a newer one (79); no audit record (93); task or sprint not found (120, 146, 188); project not loaded (235). **Sprint selection:** the first sprint created becomes active (165); starting a sprint selects it (178); deleting the active sprint clears the selection (206); selecting an unknown sprint is ignored (223).
`services/flowHistoryStore.js`	0.0% (0/23)	No test imports the module (Section 6.3).

File	Coverage	Missed outcomes, by behaviour
services/feedbackStore.js	20.0% (2/10)	The "Completed" versus "Saved" versus "Created" wording, and the haptics options, in announceSaved (lines 11–12).
utils/taskForm.js	80.0% (24/30)	Defaults for a task missing its title, status, priority or estimate (lines 34–38); a new task when no sprint is selected (52); labels that are not a list (14).
utils/taskExplorer.js	91.2% (62/68)	Search across tasks without a description, assignee or labels (9); due-date sorting when both tasks have no date (12); CSV export of a task without a sprint or labels (26).
utils/auditTrail.js	92.1% (58/63)	An empty list change shown as "—" (57); a record without a timestamp (188); one side of the "N hours ago" wording (193).

Most missed outcomes in the logic layer fall into two groups: defensive fallbacks for malformed or missing data, which never run because the seed data and test fixtures are always well formed; and cloud-failure paths, which need a mocked failure for each request. Neither group is reached in normal use, and that is exactly why defects tend to hide there: they only run once something has already gone wrong.

12. Branch Coverage of User Interface Files

File	Decision points	Outcomes taken	Branch coverage
App.jsx	8	10/16	62.5%
components/auth/SignInView.jsx	6	10/12	83.3%
components/dashboard/AuditTrail.jsx	10	15/20	75.0%
components/dashboard/BurndownChart.jsx	4	8/8	100.0%
components/dashboard/DashboardView.jsx	5	0/9	0.0%
components/dashboard/DeliverySignals.jsx	7	0/15	0.0%
components/dashboard/FlowAnalytics.jsx	16	0/32	0.0%
components/dashboard/KpiCards.jsx	2	4/4	100.0%
components/dashboard/ReportsWorkbench.jsx	20	0/40	0.0%
components/dashboard/WorkloadList.jsx	1	2/2	100.0%
components/docs/DocsView.jsx	6	0/12	0.0%
components/kanban/KanbanBoard.jsx	15	18/32	56.3%
components/kanban/LabelPills.jsx	1	2/2	100.0%
components/kanban/TaskModal.jsx	18	18/37	48.6%
components/layout/Navbar.jsx	11	19/23	82.6%
components/layout/Sidebar.jsx	2	4/4	100.0%
components/settings/ProjectSettingsModal.jsx	12	19/23	82.6%
components/settings/SettingsModal.jsx	20	12/42	28.6%
components/sprint/SprintManager.jsx	13	0/27	0.0%
components/workspace/CommandMenu.jsx	16	6/32	18.8%
components/workspace/ErrorBoundary.jsx	1	0/2	0.0%
components/workspace/ProjectMap.jsx	33	0/67	0.0%
components/workspace/SignalIcon.jsx	2	2/3	66.7%
components/workspace/TaskExplorer.jsx	17	0/35	0.0%
components/workspace/TaskPreviewDrawer.jsx	17	0/35	0.0%
components/workspace/TaskSignals.jsx	6	9/12	75.0%
components/workspace/WorkSignals.jsx	5	2/10	20.0%

File	Decision points	Outcomes taken	Branch coverage
components/workspace/WorkspaceFeedback.jsx	18	4/37	10.8%

Unit-level branch coverage of the components is low for the reasons in Section 6.2. Most of these branches decide what to render, and they are exercised by the system test cases in the System Validation Report. Branch counters are not collected from the browser runs, so those executions do not appear in these figures.

13. Recommendations (Branch Coverage)

Priority	Test to add	Outcomes it closes
1	estimation: a sprint containing a task with no story points and one with a non-numeric value	5 outcomes; estimation.js to 100%
2	supabaseSync: mock a failing sprint delete and a failing audit insert while the queue drains; assert the change stays queued for retry	Sync-queue failure exits (lines 447–461, 584)
3	supabaseSync: load audit rows with no metadata, and with details that are not JSON	Older-row parsing (lines 199–227)
4	ProjectContext: delete the active sprint, select a sprint that no longer exists, and resolve two loads out of order	The guard and sprint-selection outcomes in Section 11.1
5	New flowHistoryStore.test.js: empty and corrupt storage, one observation per day, cache reuse	23 outcomes
6	Component tests (React Testing Library) for open, closed and error states of TaskModal, SettingsModal and SprintManager	Most of the 47% ternary figure
7	CI gate: run coverage in CI with branch thresholds of 90% for src/utils and 80% for src/services	Keeps the logic layer from regressing

Recommendations 1 to 4 are small, self-contained tests. Together they would lift src/services above 80% and the logic layer to roughly 90% branch coverage.

14. Conclusion

The STP coverage target is met: the utility modules have 99.3% line coverage and 92.9% branch coverage against a target of 80%. Across the whole logic layer, utilities and services together, line coverage is 95.2% and branch coverage 87.9%. The whole-application figures (70.5% of lines, 64.1% of branch outcomes) are lower because most React view components are tested in the browser rather than in the unit suite. The clearest gaps are services/flowHistoryStore.js, which no test imports, and the failure paths of the Supabase sync queue. Sections 7 and 13 list the tests that would close them.

Appendix: Reproducing These Figures

```
npm ci
npm run test:coverage      # text summary in the terminal; HTML report in coverage/index.html
# branch totals: the "% Branch" column, or coverage/coverage-summary.json
# missed outcomes: open coverage/index.html; untaken branches are highlighted in yellow
```